
Diseño Arquitectónico

Sistema de Banca por Internet BP

Evaluación Técnica

Arquitecto de Soluciones / Líder Técnico de Desarrollo

Autor: Ing. David A. Palacios

Fecha: 11 De Junio Del 2026

Versión 1.1.1

ÍNDICE

ÍNDICE.....	2
1. Introducción.....	6
1.1 Objetivo del documento.....	6
1.2 Alcance.....	6
1.3 Supuestos.....	7
1.4 Restricciones.....	7
1.5 Audiencia objetivo.....	7
2. Análisis del Problema.....	8
2.1 Contexto de Negocio.....	8
2.2 Objetivos de Negocio.....	8
2.3 Actores del Sistema.....	9
2.4 Requerimientos Funcionales.....	9
2.5 Requerimientos No Funcionales.....	12
2.6 Drivers Arquitectónicos.....	12
2.7 Riesgos Arquitectónicos Identificados.....	13
2.8 Quality Attribute Scenarios (QAS).....	14
3. Decisiones Arquitectónicas (ADR).....	14
ADR-001 - Arquitectura basada en Microservicios.....	14
ADR-002 - Arquitectura Event-Driven.....	16
ADR-003 - AWS como Plataforma Cloud Principal.....	17
ADR-004 - API Gateway como Punto Único de Entrada.....	18
ADR-005 - Patrón Backend For Frontend (BFF).....	18
ADR-006 OAuth 2.0 + OpenID Connect.....	19
ADR-007 - Authorization Code Flow con PKCE.....	19
ADR-008 - Keycloak como Identity Provider.....	19
ADR-009 - Angular para la SPA.....	20
ADR-010 - React Native para Aplicación Móvil.....	20
ADR-011 - PostgreSQL como Base de Datos Transaccional Principal.....	21
ADR-012 - Redis y Patrón Cache-Aside para Optimización de Consultas.....	22
ADR-013 - CQRS + Event Sourcing Parcial + Outbox Pattern.....	23
ADR-014 - Observabilidad basada en OpenTelemetry, Prometheus y Grafana.....	24
ADR-015 - Kubernetes (Amazon EKS) como Plataforma de Orquestación.....	25
4. Modelo C4.....	27
4.1 Introducción al Modelo C4.....	27
4.2 Diagrama de Contexto (Nivel 1).....	27
Objetivo.....	27
Alcance.....	28
Diagrama.....	28
Distribución visual.....	29
Elementos del Diagrama.....	29
Relaciones.....	30
Justificación Arquitectónica del Context Diagram.....	31

Relación con ADRs.....	31
4.3 Diagrama de Contenedores (Nivel 2).....	32
Objetivo.....	32
Diagrama.....	33
Agrupación visual.....	34
Contenedores.....	35
Relaciones.....	35
Relación con ADRs.....	36
4.4 Diagrama de Componentes (Nivel 3).....	36
Objetivo.....	36
Diagrama.....	37
Patrones aplicados.....	38
Relación con ADR.....	38
5. Arquitectura de Seguridad y Autenticación.....	39
5.1 Objetivo.....	39
5.2 Arquitectura de Autenticación.....	39
Justificación Arquitectónica.....	39
Riesgos Mitigados.....	40
Beneficios.....	40
5.3 Flujo de Autenticación Web.....	40
5.4 Flujo de Autenticación Mobile.....	40
5.5 Multi-Factor Authentication (MFA).....	41
5.6 Passkeys y FIDO2.....	41
5.7 Gestión de Sesiones.....	41
5.8 Controles de Seguridad.....	42
5.9 OWASP y Seguridad Aplicativa.....	42
5.10 Diagrama de Flujo de Autenticación.....	43
6. Arquitectura de Onboarding Biométrico.....	43
6.1 Objetivo.....	43
6.2 Flujo de Onboarding.....	43
6.3 Componentes del Flujo.....	44
Justificación Arquitectónica.....	44
6.4 Herramientas Evaluadas.....	45
6.5 Decisión.....	46
6.6 Diagrama de Proceso Onboarding Biométrico.....	46
7. Auditoría y Persistencia de Clientes Frecuentes.....	47
7.1 Arquitectura de Auditoría.....	47
7.2 Flujo de Auditoría.....	47
7.3 Justificación.....	47
7.4 Persistencia de Clientes Frecuentes.....	47
7.5 Diagramas.....	48
8. Infraestructura AWS.....	49
Alta Disponibilidad (HA).....	50
Recuperación ante Desastres (DR).....	50

Auto-Healing.....	51
Diagrama AWS.....	51
9. Regulaciones y Cumplimiento.....	52
9.1 Objetivo.....	52
9.2 Marco Regulatorio Aplicable.....	52
9.3 Estándares Técnicos de Seguridad.....	53
9.4 Gobierno y Gestión de Riesgos.....	53
9.5 Cumplimiento y Trazabilidad.....	53
9.6 Justificación Arquitectónica.....	54
9.7 Riesgos Mitigados.....	54
9.8 Beneficios.....	54
10. Operación, Monitoreo y DevOps.....	54
10.1 Objetivo.....	54
10.2 Observabilidad.....	54
Métricas.....	54
Logs.....	55
Trazas Distribuidas.....	55
10.3 Herramientas de Monitoreo.....	55
10.4 Indicadores Operativos.....	55
10.5 Gestión de Incidentes.....	55
10.6 Estrategia DevOps.....	56
10.7 Pipeline CI/CD.....	56
10.8 Controles de Calidad.....	56
10.9 Estrategias de Despliegue.....	56
10.10 Infraestructura como Código (IaC).....	57
10.11 Justificación Arquitectónica.....	57
10.12 Beneficios.....	57
11. Costos y Optimización.....	57
11.1 Objetivo.....	57
11.2 Principios de Optimización.....	57
11.3 Justificación de Servicios Administrados.....	58
11.4 Estrategias de Optimización.....	58
Auto Scaling.....	58
Ambientes Independientes.....	58
Uso de Contenedores.....	58
Almacenamiento por Ciclo de Vida.....	58
11.5 Evaluación de Alternativas.....	59
Opción 1: Infraestructura Tradicional.....	59
Opción 2: Servicios Administrados AWS.....	59
11.6 FinOps y Gobierno de Costos.....	59
11.7 Balance entre Costo y Disponibilidad.....	60
11.8 Justificación Arquitectónica.....	60
11.9 Beneficios.....	60
12. Matriz de Cobertura de Requerimientos.....	61

12.1 Objetivo.....	61
12.2 Cobertura de Requerimientos Funcionales.....	61
12.3 Cobertura de Requerimientos No Funcionales.....	62
12.4 Cobertura de Requerimientos Tecnológicos.....	62
12.5 Cobertura de Criterios de Evaluación.....	63
12.6 Resultado de Validación.....	63
13. Conclusiones Arquitectónicas.....	64
14. Referencias y Fuentes Consultadas.....	64

1. Introducción

1.1 Objetivo del documento

El presente documento tiene como objetivo definir la arquitectura de solución para la plataforma de Banca por Internet de la entidad BP, proporcionando una visión integral de los componentes funcionales, tecnológicos y operativos necesarios para soportar los servicios digitales ofrecidos a sus clientes.

La propuesta arquitectónica busca satisfacer los requerimientos funcionales planteados para consulta de movimientos, transferencias, pagos y onboarding digital, así como los requerimientos no funcionales asociados a disponibilidad, seguridad, resiliencia, trazabilidad, cumplimiento regulatorio y escalabilidad.

Adicionalmente, este documento presenta el diseño arquitectónico siguiendo el modelo C4 (Contexto, Contenedores y Componentes), incorporando principios de arquitectura moderna basados en microservicios, eventos, observabilidad y seguridad por diseño (Security by Design).

1.2 Alcance

La arquitectura propuesta cubre los siguientes dominios funcionales:

- Consulta de información de clientes.
- Consulta de productos bancarios.
- Consulta de movimientos históricos.
- Transferencias entre cuentas propias.
- Transferencias interbancarias.
- Pago de servicios y terceros.
- Onboarding digital con validación biométrica.
- Gestión de autenticación y autorización.
- Gestión de auditoría y trazabilidad.
- Gestión de notificaciones regulatorias.
- Integración con sistemas internos y externos.

La propuesta contempla además componentes de infraestructura cloud, mecanismos de alta disponibilidad, recuperación ante desastres, monitoreo, observabilidad y controles de seguridad requeridos para una entidad financiera moderna.

1.3 Supuestos

Para el diseño de la solución se consideran los siguientes supuestos:

- La entidad BP dispone de un Core Bancario que expone servicios de integración para consulta y ejecución de transacciones.
- Existe un sistema complementario que provee información detallada de clientes.
- La organización cuenta con un proveedor de identidad configurable compatible con OAuth 2.0 y OpenID Connect.
- Existe presupuesto suficiente para utilizar servicios administrados en la nube.
- Las integraciones con ACH, BCE y pasarelas de pago disponen de interfaces de comunicación seguras y documentadas.
- Los canales digitales serán utilizados por clientes de banca personal inicialmente, con posibilidad de expansión futura hacia banca empresarial.

1.4 Restricciones

Las siguientes restricciones fueron identificadas a partir del enunciado:

- Debe implementarse el modelo C4 hasta nivel de Componentes.
- Deben existir al menos dos mecanismos de notificación.
- Debe utilizarse OAuth 2.0 como mecanismo principal de autenticación.
- Debe contemplarse onboarding con reconocimiento facial.
- Debe existir un mecanismo de auditoría integral.
- Debe garantizarse alta disponibilidad, resiliencia y recuperación ante desastres.
- La solución debe ser escalable y preparada para el crecimiento futuro.

1.5 Audiencia objetivo

Este documento está dirigido a:

- Arquitectos de Solución.
- Arquitectos Empresariales.
- Arquitectos Cloud.
- Líderes Técnicos.
- Product Owners.
- Equipos de Desarrollo.
- Equipos de Seguridad.
- Equipos de Infraestructura.
- Auditores internos y externos.
- Stakeholders de negocio.

2. Análisis del Problema

2.1 Contexto de Negocio

La entidad financiera BP requiere implementar una plataforma de Banca por Internet moderna que permita a sus clientes realizar operaciones financieras a través de canales digitales web y móviles.

Actualmente la información necesaria para atender las operaciones bancarias se encuentra distribuida entre múltiples sistemas empresariales. El sistema principal corresponde al Core Bancario, encargado de gestionar clientes, cuentas, productos y movimientos financieros, mientras que un sistema complementario provee información adicional requerida para determinados procesos de consulta y validación.

La solución propuesta debe ofrecer una experiencia digital segura, consistente y omnicanal, permitiendo a los clientes realizar consultas, transferencias, pagos y procesos de onboarding digital sin necesidad de acudir a una sucursal física.

Desde la perspectiva empresarial, la plataforma deberá convertirse en un habilitador estratégico para la transformación digital de la institución financiera, reduciendo costos operativos, incrementando la adopción de canales digitales y fortaleciendo los mecanismos de seguridad, trazabilidad y cumplimiento regulatorio.

2.2 Objetivos de Negocio

La arquitectura propuesta debe contribuir al cumplimiento de los siguientes objetivos estratégicos:

ID	Objetivo
OBJ-01	Permitir a los clientes consultar información financiera desde cualquier canal digital.
OBJ-02	Habilitar transferencias entre cuentas propias y cuentas de terceros.
OBJ-03	Facilitar transferencias interbancarias mediante integración con redes financieras externas.
OBJ-04	Permitir pagos digitales a servicios, comercios y terceros.
OBJ-05	Incorporar nuevos clientes mediante procesos de onboarding digital.
OBJ-06	Reducir la dependencia de canales presenciales.
OBJ-07	Incrementar la satisfacción y experiencia del cliente.
OBJ-08	Garantizar trazabilidad completa de todas las operaciones.

OBJ-09	Cumplir normativas financieras y regulatorias.
OBJ-10	Disponer de una arquitectura escalable preparada para futuros productos digitales.

2.3 Actores del Sistema

- **Cliente Existente**

Persona que posee productos financieros activos y utiliza la plataforma para consultar información, realizar transferencias y efectuar pagos.

- **Cliente Nuevo**

Persona que aún no posee acceso a la plataforma y requiere completar un proceso de onboarding digital con validación biométrica para habilitar sus credenciales.

- **Administrador Operativo**

Usuario interno responsable de monitorear el funcionamiento de la plataforma, revisar eventos de auditoría y gestionar incidencias operativas.

- **Equipo de Soporte**

Responsable de brindar soporte técnico, análisis de incidencias y atención de eventos críticos relacionados con la operación de la plataforma.

- **Sistemas Externos**

Incluyen el Core Bancario, sistema complementario de clientes, redes interbancarias, motores biométricos, proveedores de notificaciones y sistemas regulatorios.

2.4 Requerimientos Funcionales

- **Gestión de Clientes**

ID	Requerimiento
RF-01	Consultar información básica del cliente.
RF-02	Consultar productos financieros asociados al cliente.
RF-03	Consultar información ampliada proveniente del sistema complementario.

- **Consulta de Información Financiera**

ID	Requerimiento
RF-04	Consultar saldo de cuentas.
RF-05	Consultar movimientos históricos.
RF-06	Filtrar movimientos por rango de fechas.
RF-07	Consultar detalles de una transacción específica.

- **Transferencias**

ID	Requerimiento
RF-08	Realizar transferencias entre cuentas propias.
RF-09	Realizar transferencias entre cuentas del mismo banco.
RF-10	Realizar transferencias interbancarias.
RF-11	Validar saldo disponible antes de ejecutar una transferencia.
RF-12	Notificar el resultado de la transferencia al cliente.

- **Pagos**

ID	Requerimiento
RF-13	Realizar pagos a servicios.
RF-14	Realizar pagos a terceros.
RF-15	Confirmar y registrar pagos efectuados.

- **Autenticación y Seguridad**

ID	Requerimiento
RF-16	Permitir autenticación mediante usuario y contraseña.
RF-17	Permitir autenticación biométrica mediante huella o reconocimiento facial del dispositivo.
RF-18	Gestionar sesiones seguras mediante OAuth 2.0 y OpenID Connect.
RF-19	Aplicar autenticación reforzada para operaciones sensibles.

- **Onboarding Digital**

ID	Requerimiento
RF-20	Registrar nuevos clientes mediante onboarding digital.
RF-21	Validar identidad mediante reconocimiento facial.
RF-22	Capturar documentos de identidad.
RF-23	Realizar validaciones de liveness detection.
RF-24	Crear credenciales de acceso para nuevos usuarios.

- **Notificaciones**

ID	Requerimiento
RF-25	Enviar notificaciones por correo electrónico.
RF-26	Enviar notificaciones por SMS.
RF-27	Enviar notificaciones push a dispositivos móviles.
RF-28	Confirmar entrega de notificaciones.

- **Auditoría**

ID	Requerimiento
RF-29	Registrar todos los eventos de autenticación.
RF-30	Registrar consultas realizadas por los clientes.
RF-31	Registrar transferencias y pagos.
RF-32	Mantener trazabilidad completa de las operaciones.

2.5 Requerimientos No Funcionales

Categoría	Requerimiento
Disponibilidad	Disponibilidad mínima de 99.95%.
Rendimiento	Consultas inferiores a 300 ms para operaciones cacheadas.
Rendimiento	Operaciones transaccionales inferiores a 3 segundos.
Escalabilidad	Escalamiento horizontal de componentes críticos.
Resiliencia	Tolerancia a fallos sin interrupción del servicio.
Recuperación	RTO menor o igual a 1 hora.
Recuperación	RPO menor o igual a 15 minutos.
Seguridad	Cifrado TLS 1.3 en tránsito.
Seguridad	Cifrado AES-256 en reposo.
Seguridad	Autenticación multifactor.
Auditoría	Registro de todas las operaciones ejecutadas.
Observabilidad	Monitoreo centralizado de logs, métricas y trazas.
Operación	Despliegues automatizados mediante CI/CD.
Mantenibilidad	Arquitectura desacoplada y extensible.
Cumplimiento	Cumplimiento de PCI DSS, ISO 27001 y LOPDP.

2.6 Drivers Arquitectónicos

Los drivers arquitectónicos representan aquellos factores que influyen directamente sobre las decisiones de diseño.

- **Driver 1: Seguridad**

Debido a la naturaleza financiera de la solución, la seguridad se convierte en el principal driver arquitectónico. La plataforma deberá proteger información sensible, prevenir accesos no autorizados, garantizar autenticación robusta y mantener la integridad de las transacciones.

- **Driver 2: Disponibilidad**

Los servicios financieros digitales deben permanecer operativos de forma continua. Una interrupción afecta directamente la confianza del cliente y puede generar impactos económicos y regulatorios.

- **Driver 3: Escalabilidad**

La plataforma debe soportar incrementos significativos de usuarios, transacciones y productos digitales sin requerir rediseños arquitectónicos.

- **Driver 4: Cumplimiento Regulatorio**

La arquitectura debe incorporar controles que permitan cumplir con regulaciones financieras, requisitos de auditoría y protección de datos personales.

- **Driver 5: Integración**

La solución dependerá de múltiples sistemas internos y externos, por lo que deberá facilitar integraciones desacopladas y resilientes.

- **Driver 6: Observabilidad**

La operación bancaria exige monitoreo continuo, trazabilidad completa y capacidad de diagnóstico ante incidentes.

2.7 Riesgos Arquitectónicos Identificados

ID	Riesgo	Impacto
RA-01	Indisponibilidad del Core Bancario	Alto
RA-02	Saturación por crecimiento de usuarios	Alto
RA-03	Fallos en proveedores externos de notificaciones	Medio
RA-04	Fraude mediante robo de credenciales	Alto
RA-05	Fallos en integración con ACH o BCE	Alto
RA-06	Pérdida de eventos de auditoría	Crítico
RA-07	Exposición de información biométrica	Crítico
RA-08	Dependencia excesiva de componentes monolíticos	Alto
RA-09	Latencia excesiva en consultas al Core	Alto
RA-10	Incumplimiento regulatorio	Crítico

2.8 Quality Attribute Scenarios (QAS)

- **Escenario de Disponibilidad**

Cuando falle una instancia de un microservicio en producción, el sistema deberá recuperarse automáticamente sin intervención manual y sin afectar la continuidad del servicio para el usuario final.

- **Escenario de Rendimiento**

Cuando la plataforma reciba un incremento repentino del 300% en consultas de movimientos, el tiempo promedio de respuesta deberá mantenerse por debajo de un segundo mediante mecanismos de cacheo y escalamiento automático.

- **Escenario de Seguridad**

Cuando un usuario intente acceder a información sensible desde un dispositivo desconocido, el sistema deberá exigir autenticación reforzada mediante MFA antes de permitir la operación.

- **Escenario de Auditoría**

Cuando un auditor solicite el historial de una operación financiera, deberá ser posible reconstruir completamente la secuencia de eventos asociados a dicha operación.

- **Escenario de Recuperación**

Cuando ocurra una caída total de la región principal de nube, la plataforma deberá restablecer operaciones desde la región secundaria dentro del RTO y RPO definidos.

- **Escenario de Escalabilidad**

Cuando el volumen de usuarios concurrentes se duplique durante campañas comerciales o fechas de alta demanda, la plataforma deberá escalar horizontalmente sin afectar la experiencia del cliente.

3. Decisiones Arquitectónicas (ADR)

Las siguientes Architectural Decision Records (ADR) documentan las principales decisiones de diseño adoptadas para la solución propuesta.

Su objetivo es proporcionar trazabilidad arquitectónica, justificar la selección tecnológica realizada y facilitar futuras evoluciones de la plataforma.

ADR-001 - Arquitectura basada en Microservicios

Estado: Aceptado

Contexto:

La plataforma de banca digital deberá soportar múltiples dominios funcionales independientes, incluyendo autenticación, consulta de movimientos, transferencias, pagos, onboarding, auditoría y notificaciones.

Cada dominio posee patrones de consumo distintos, diferentes niveles de criticidad y requerimientos de escalabilidad particulares.

Además, la entidad financiera requiere una arquitectura preparada para incorporar nuevos productos digitales en el futuro sin afectar la estabilidad de los servicios existentes.

Alternativas Evaluadas

- Monolito tradicional.
- Monolito modular.
- Arquitectura orientada a servicios (SOA).
- Arquitectura basada en microservicios.

Decisión

Se adopta una arquitectura basada en microservicios.

Justificación

- **Justificación 1: Escalabilidad independiente por dominio**
Los diferentes procesos bancarios presentan comportamientos de carga distintos. Por ejemplo, las consultas de movimientos pueden representar decenas de veces más tráfico que las transferencias o pagos.
Una arquitectura de microservicios permite escalar únicamente los servicios que experimentan mayor demanda, optimizando recursos de infraestructura y reduciendo costos operativos.
- **Justificación 2: Aislamiento de fallos**
En sistemas financieros es fundamental evitar que la falla de un componente afecte al resto de la plataforma.
Mediante microservicios es posible aplicar mecanismos de resiliencia específicos por dominio (Circuit Breaker, Retry, Bulkhead), limitando la propagación de errores y mejorando la disponibilidad global del sistema.
- **Justificación 3: Evolución tecnológica independiente**
La plataforma deberá evolucionar durante varios años. Los microservicios permiten modificar, reemplazar o evolucionar componentes individuales sin requerir despliegues masivos ni afectar otros dominios de negocio.

Consecuencias Positivas

- Mayor escalabilidad.
- Mejor desacoplamiento.
- Mayor resiliencia.
- Despliegues independientes.

Consecuencias Negativas

- Mayor complejidad operativa.
- Requiere observabilidad avanzada.
- Incrementa la complejidad de pruebas distribuidas.

ADR-002 - Arquitectura Event-Driven

Estado: Aceptado

Contexto:

La plataforma debe procesar eventos de negocio como transferencias, pagos, autenticaciones, onboarding y auditoría. Adicionalmente, múltiples componentes necesitan reaccionar a los mismos eventos sin generar dependencias directas.

Alternativas Evaluadas:

- Comunicación exclusivamente síncrona REST.
- RabbitMQ.
- AWS SNS/SQS.
- Arquitectura basada en eventos con Kafka.

Decisión

Adoptar una arquitectura Event-Driven soportada por Apache Kafka (AWS MSK).

Justificación

- **Justificación 1: Desacoplamiento entre dominios**

Cuando una transferencia es completada, múltiples sistemas necesitan reaccionar:

- Auditoría.
- Notificaciones.
- Monitoreo.
- Analítica.

Kafka permite que cada consumidor procese el evento de forma independiente sin afectar al productor original.

- **Justificación 2: Persistencia y trazabilidad de eventos**

A diferencia de mecanismos tradicionales de mensajería, Kafka conserva los eventos durante períodos configurables. Esto facilita auditorías regulatorias, reconstrucción de operaciones y análisis posteriores.

- **Justificación 3: Escalabilidad horizontal**

Kafka permite distribuir cargas entre múltiples particiones y consumidores, soportando altos volúmenes transaccionales sin afectar la estabilidad de la plataforma.

Consecuencias Positivas:

- Desacoplamiento.
- Escalabilidad.
- Trazabilidad.

Consecuencias Negativas

- Mayor complejidad operativa.
- Curva de aprendizaje superior.
- Necesidad de gobernanza de eventos.

ADR-003 - AWS como Plataforma Cloud Principal

Estado: Aceptado

Contexto:

La solución requiere alta disponibilidad, recuperación ante desastres, escalabilidad automática, servicios administrados y cumplimiento regulatorio.

Alternativas Evaluadas

- Infraestructura On-Premise.
- Microsoft Azure.
- Google Cloud Platform.
- Amazon Web Services.

Decisión: Adoptar AWS como plataforma cloud principal.

Justificación

- **Justificación 1: Madurez en el sector financiero**
AWS posee amplia adopción en instituciones financieras globales y regionales, además de certificaciones como PCI DSS, ISO 27001, SOC 1, SOC 2 y SOC 3. Esto reduce riesgos de cumplimiento y facilita auditorías regulatorias.
- **Justificación 2: Amplio catálogo de servicios administrados**
Servicios como EKS, RDS, MSK, ElastiCache y CloudWatch permiten reducir significativamente la carga operativa sobre equipos de infraestructura. Esto permite concentrar esfuerzos en capacidades de negocio y no en administración de plataformas.
- **Justificación 3: Alta disponibilidad nativa**
AWS proporciona capacidades Multi-AZ, Auto Scaling y recuperación automatizada, fundamentales para alcanzar los objetivos de disponibilidad definidos para la plataforma.

Consecuencias Positivas

- Menor operación.
- Alta disponibilidad.
- Escalabilidad nativa.

Consecuencias Negativas

- Dependencia del proveedor.
- Costos de salida.
- Necesidad de gobierno cloud.

ADR-004 - API Gateway como Punto Único de Entrada

Estado: Aceptado

Alternativas Evaluadas

- Acceso directo a microservicios.
- Reverse Proxy tradicional.
- API Gateway.

Decisión: Implementar API Gateway como punto único de acceso.

Justificación

- **Justificación 1:** Centraliza autenticación, autorización, rate limiting y validación de tokens.
- **Justificación 2:** Reduce la exposición directa de microservicios a Internet.
- **Justificación 3:** Permite aplicar políticas de seguridad y observabilidad de forma uniforme en toda la plataforma.

Consecuencias Positivas

- Seguridad centralizada.
- Menor acoplamiento.
- Gobierno de APIs.

Consecuencias Negativas

- Posible cuello de botella si no se diseña adecuadamente.

ADR-005 - Patrón Backend For Frontend (BFF)

Decisión: Implementar un BFF para Web y un BFF para Mobile.

Justificación

- **Justificación 1:** Los canales web y móvil poseen necesidades distintas de presentación y agregación de información.
- **Justificación 2:** Permite optimizar payloads y reducir llamadas desde dispositivos móviles.
- **Justificación 3:** Evita que cambios en microservicios impacten directamente los canales digitales.

ADR-006 OAuth 2.0 + OpenID Connect

Decisión: Implementar OAuth 2.0 y OpenID Connect.

Justificación

- **Justificación 1:** OAuth 2.0 es el estándar predominante para autorización segura en aplicaciones modernas.
- **Justificación 2:** OpenID Connect extiende OAuth proporcionando identidad verificable mediante tokens estandarizados.
- **Justificación 3:** Permite integrar mecanismos avanzados como MFA, biometría y Passkeys sin modificar los sistemas consumidores.

ADR-007 - Authorization Code Flow con PKCE

Decisión: Utilizar Authorization Code Flow con PKCE para SPA y Mobile.

Justificación

- **Justificación 1:** Mitiga ataques de interceptación del código de autorización.
- **Justificación 2:** Elimina la necesidad de almacenar Client Secrets en aplicaciones públicas.
- **Justificación 3:** Es el flujo recomendado actualmente por OAuth 2.1 y OWASP ASVS.

ADR-008 - Keycloak como Identity Provider

Decisión: Utilizar Keycloak como proveedor de identidad.

Justificación

- **Justificación 1:** Implementa OAuth2, OIDC, SAML y FIDO2 de forma nativa.
- **Justificación 2:** Permite personalizar flujos de autenticación para onboarding biométrico.
- **Justificación 3:** Evita costos por usuario asociados a soluciones comerciales.

ADR-009 - Angular para la SPA

Decisión: Seleccionar Angular para el canal web.

Justificación

- **Justificación 1:**
Angular incorpora mecanismos nativos de protección contra Cross-Site Scripting (XSS) mediante sanitización automática del DOM, reduciendo significativamente la superficie de ataque frente a aplicaciones financieras expuestas a Internet.
- **Justificación 2:**
La arquitectura basada en módulos, componentes, servicios e inyección de dependencias favorece el mantenimiento a largo plazo y reduce la deuda técnica en proyectos empresariales complejos.
- **Justificación 3:**
Su integración nativa con TypeScript facilita la construcción de aplicaciones robustas, permitiendo validaciones tempranas, mejor mantenibilidad y menor probabilidad de errores en producción.

ADR-010 - React Native para Aplicación Móvil

Decisión: Seleccionar React Native como framework multiplataforma.

Justificación

- **Justificación 1:**
Permite reutilizar conocimientos y prácticas de desarrollo basadas en JavaScript y TypeScript, reduciendo tiempos de capacitación y facilitando la incorporación de nuevos desarrolladores.
- **Justificación 2:**
Ofrece acceso maduro a capacidades biométricas nativas mediante integración con Face ID, Touch ID y Android Biometric Authentication, fundamentales para procesos de autenticación bancaria.
- **Justificación 3:**
Reduce el costo total de propiedad al mantener una única base de código para iOS y Android, disminuyendo esfuerzos de mantenimiento y evolución funcional.

ADR-011 - PostgreSQL como Base de Datos Transaccional Principal

Estado: Aceptado

Contexto:

La plataforma de banca digital requiere almacenar información transaccional relacionada con transferencias, pagos, onboarding, auditoría operacional y configuraciones del sistema.

Los datos gestionados poseen características altamente transaccionales, requieren consistencia fuerte y deben garantizar integridad ante fallos parciales, errores de red o concurrencia elevada. Adicionalmente, la solución debe soportar mecanismos de replicación, recuperación ante desastres, cifrado y auditoría.

Alternativas Evaluadas

- Oracle Database.
- Microsoft SQL Server.
- MySQL.
- PostgreSQL.

Decisión: Adoptar PostgreSQL como motor principal para la persistencia transaccional.

Justificación

• **Justificación 1: Cumplimiento de propiedades ACID**

Las operaciones bancarias requieren garantizar atomicidad, consistencia, aislamiento y durabilidad.

PostgreSQL implementa completamente las propiedades ACID, permitiendo asegurar que operaciones críticas como transferencias, pagos y actualizaciones de saldo mantengan integridad incluso ante fallos de infraestructura o interrupciones inesperadas.

• **Justificación 2: Capacidades empresariales avanzadas sin costos de licenciamiento**

PostgreSQL ofrece replicación, particionamiento, cifrado, control granular de acceso, procedimientos almacenados y alta disponibilidad sin requerir licencias propietarias. Esto permite construir una solución empresarial robusta reduciendo costos operativos y dependencia tecnológica.

• **Justificación 3: Amplia adopción en entornos financieros**

PostgreSQL es ampliamente utilizado en instituciones financieras, fintechs y plataformas de pagos debido a su estabilidad, rendimiento y madurez.

Además, cuenta con soporte nativo en AWS RDS, facilitando operaciones de backup, recuperación, monitoreo y escalabilidad.

Consecuencias Positivas

- Consistencia transaccional.
- Alta confiabilidad.

- Menor costo de licenciamiento.
- Compatibilidad con servicios administrados.

Consecuencias Negativas

- Escalabilidad horizontal más compleja que bases NoSQL.
- Requiere estrategias específicas para particionamiento y crecimiento masivo.

ADR-012 - Redis y Patrón Cache-Aside para Optimización de Consultas

Estado: Aceptado

Contexto:

La mayoría de las operaciones realizadas por los clientes corresponden a consultas de información como:

- Productos.
- Perfil.
- Saldos.
- Movimientos recientes.

Estas consultas generan una carga significativa sobre los sistemas Core, los cuales normalmente poseen costos elevados de procesamiento y tiempos de respuesta superiores a los requeridos por los canales digitales.

Alternativas Evaluadas

- Sin mecanismo de caché.
- Caché local en aplicación.
- Caché distribuida Redis.
- Base de datos NoSQL como caché principal.

Decisión: Implementar Redis utilizando el patrón Cache-Aside.

Justificación

- **Justificación 1: Reducción significativa de latencia**

Las consultas frecuentes pueden resolverse directamente desde memoria sin necesidad de acceder continuamente al Core Bancario.

Esto permite reducir tiempos de respuesta desde cientos de milisegundos o segundos hasta decenas de milisegundos, mejorando significativamente la experiencia del usuario.

- **Justificación 2: Disminución de carga sobre sistemas legados**

Los sistemas Core suelen ser recursos críticos y costosos. Redis permite reducir la cantidad de consultas realizadas a dichos sistemas, disminuyendo costos operativos y evitando cuellos de botella durante períodos de alta demanda.

- **Justificación 3: Control explícito sobre la estrategia de cacheo**

El patrón Cache-Aside permite que la aplicación controle cuándo almacenar, invalidar o actualizar información.

Esto proporciona mayor precisión y consistencia respecto a mecanismos automáticos de caché menos controlables.

Consecuencias Positivas

- Menor latencia.
- Menor carga sobre el Core.
- Mejor experiencia de usuario.
- Escalabilidad de lectura.

Consecuencias Negativas

- Posible inconsistencia temporal de datos.
- Necesidad de estrategias de invalidación adecuadas.

ADR-013 - CQRS + Event Sourcing Parcial + Outbox Pattern

Estado: Aceptado

Contexto:

La plataforma debe satisfacer simultáneamente requerimientos de:

- Alto volumen de consultas.
- Auditoría completa.
- Trazabilidad regulatoria.
- Integración desacoplada entre componentes.

Las necesidades de lectura son considerablemente superiores a las necesidades de escritura, especialmente para movimientos históricos y consultas de clientes.

Alternativas Evaluadas

- CRUD tradicional.
- Event Sourcing completo.
- CQRS simple.
- CQRS + Event Sourcing parcial + Outbox Pattern.

Decisión: Implementar CQRS complementado con Event Sourcing parcial para auditoría y Outbox Pattern para garantizar consistencia en la publicación de eventos.

Justificación

- **Justificación 1: Separación eficiente entre lectura y escritura**

Las operaciones de lectura y escritura poseen comportamientos distintos.

CQRS permite optimizar cada modelo de forma independiente, mejorando rendimiento y reduciendo complejidad en consultas masivas.

- **Justificación 2: Trazabilidad completa de eventos críticos**

Las entidades financieras requieren reconstruir operaciones históricas ante auditorías, investigaciones o disputas.

Event Sourcing parcial permite conservar eventos relevantes sin necesidad de implementar dicho patrón sobre todo el dominio.

- **Justificación 3: Garantía de consistencia entre base de datos y mensajería**

El Outbox Pattern evita escenarios donde una transacción es almacenada exitosamente pero el evento asociado no es publicado.

Esto resulta crítico para procesos regulatorios, auditorías y notificaciones obligatorias.

Consecuencias Positivas

- Auditoría robusta.
- Mejor rendimiento en consultas.
- Consistencia entre eventos y datos.
- Mayor desacoplamiento.

Consecuencias Negativas

- Incrementa complejidad arquitectónica.
- Requiere gobernanza de eventos.
- Necesita monitoreo adicional.

ADR-014 - Observabilidad basada en OpenTelemetry, Prometheus y Grafana

Estado: Aceptado

Contexto:

La plataforma estará compuesta por múltiples microservicios distribuidos, integraciones externas, componentes cloud y procesos asíncronos.

La detección temprana de incidentes y el análisis de problemas requieren visibilidad completa del comportamiento del sistema.

Alternativas Evaluadas

- Logs tradicionales.
- Monitoreo exclusivo de infraestructura.
- APM propietario.
- OpenTelemetry + Prometheus + Grafana.

Decisión: Adoptar una estrategia de observabilidad basada en OpenTelemetry, Prometheus, Grafana y CloudWatch.

Justificación

- **Justificación 1: Trazabilidad distribuida extremo a extremo**

OpenTelemetry permite seguir una transacción desde el canal digital hasta los sistemas Core, incluyendo llamadas REST, eventos Kafka y procesos asíncronos. Esto reduce significativamente los tiempos de diagnóstico y resolución de incidentes.

- **Justificación 2: Estandarización y neutralidad tecnológica**

OpenTelemetry se ha convertido en el estándar de facto para la observabilidad moderna. Su adopción evita dependencias excesivas con herramientas propietarias y facilita futuras migraciones.

- **Justificación 3: Monitoreo proactivo basado en métricas de negocio**

Prometheus y Grafana permiten construir dashboards y alertas orientadas a indicadores relevantes para el negocio, como:

- Transferencias procesadas.
- Errores de autenticación.
- Tiempo promedio de respuesta.
- Disponibilidad de servicios.

Consecuencias Positivas

- Diagnóstico rápido.
- Mejor operación.
- Menor MTTR.
- Mayor visibilidad del sistema.

Consecuencias Negativas

- Incrementa almacenamiento de métricas y logs.
- Requiere gobierno sobre telemetría.

ADR-015 - Kubernetes (Amazon EKS) como Plataforma de Orquestación

Estado: Aceptado

Contexto:

La arquitectura propuesta estará compuesta por múltiples microservicios con necesidades independientes de despliegue, escalamiento y recuperación.

La organización requiere una plataforma que permita automatizar operaciones, soportar crecimiento futuro y facilitar estrategias avanzadas de despliegue.

Alternativas Evaluadas

- Máquinas virtuales tradicionales.
- AWS ECS.
- Serverless completo.
- Kubernetes (Amazon EKS).

Decisión: Adoptar Kubernetes administrado mediante Amazon EKS.

Justificación

- **Justificación 1: Escalamiento automático basado en demanda**
Kubernetes permite escalar servicios individualmente según métricas de consumo, CPU, memoria o métricas de negocio.
Esto optimiza costos y garantiza disponibilidad durante picos de carga.
- **Justificación 2: Capacidades avanzadas de resiliencia y auto-healing**
Mediante Liveness Probes, Readiness Probes, ReplicaSets y mecanismos de reconciliación automática, Kubernetes puede detectar y recuperar componentes fallidos sin intervención manual.
Esto contribuye directamente al cumplimiento de los objetivos de disponibilidad definidos para la plataforma.
- **Justificación 3: Soporte para estrategias modernas de despliegue**
Kubernetes facilita la implementación de Blue-Green Deployments, Canary Releases y Rollbacks automáticos.
Estas capacidades reducen el riesgo asociado a cambios en producción y permiten entregar nuevas funcionalidades con mayor frecuencia y seguridad.

Consecuencias Positivas

- Alta disponibilidad.
- Auto-healing.
- Escalabilidad automática.
- Despliegues avanzados.
- Portabilidad tecnológica.

Consecuencias Negativas

- Curva de aprendizaje elevada.
- Mayor complejidad operativa inicial.
- Requiere madurez DevOps y observabilidad.

4. Modelo C4

4.1 Introducción al Modelo C4

Para representar la arquitectura propuesta se utilizará el modelo C4 (Context, Containers, Components and Code), desarrollado por Simon Brown.

Este modelo permite describir la arquitectura de software mediante distintos niveles de abstracción, facilitando la comunicación entre stakeholders técnicos y no técnicos.

Los diagramas desarrollados para esta solución corresponden a:

- **Nivel 1: Diagrama de Contexto (System Context Diagram):**
Describe el ecosistema completo del sistema, mostrando actores, sistemas externos y la interacción de alto nivel con la solución propuesta.
- **Nivel 2: Container Diagram:**
Describe las principales aplicaciones, bases de datos, sistemas de mensajería y componentes tecnológicos que conforman la solución.
- **Nivel 3: Component Diagram:**
Describe los componentes internos de uno de los contenedores principales, incluyendo responsabilidades, patrones arquitectónicos y mecanismos de integración.

La utilización de este modelo permite mantener consistencia arquitectónica, trazabilidad y una visión progresiva de la solución.

4.2 Diagrama de Contexto (Nivel 1)

Objetivo

El Diagrama de Contexto representa la vista de más alto nivel de la arquitectura. Su propósito es mostrar cómo el Sistema de Banca por Internet BP interactúa con usuarios, sistemas corporativos y proveedores externos.

Este diagrama está dirigido principalmente a:

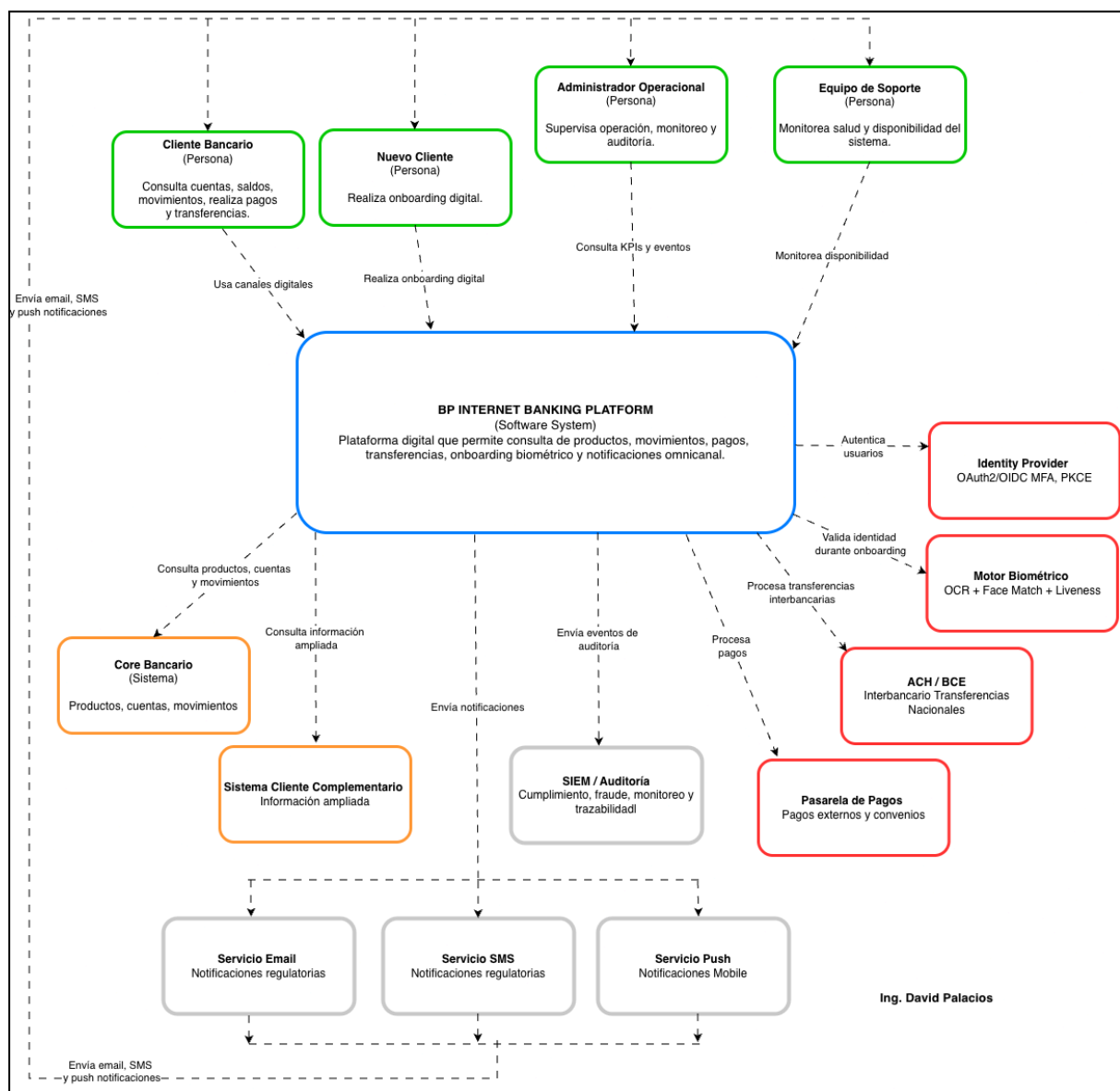
- Stakeholders de negocio.
- Product Owners.
- Arquitectos.
- Auditores.
- Equipos funcionales.
- Equipos regulatorios.

En este nivel no se representan tecnologías, protocolos, bases de datos ni detalles de implementación. El objetivo es comprender el ecosistema general en el que operará la solución.

Alcance

El sistema BP Internet Banking constituye el sistema principal dentro del alcance arquitectónico. Todos los demás sistemas representados son actores o dependencias externas con las cuales el sistema interactúa para ofrecer sus capacidades de negocio.

Diagrama



Distribución visual

- **Verde (Personas)**
 - Cliente Bancario
 - Nuevo Cliente
 - Administrador Operacional
 - Equipo de Soporte
- **Azul (Sistema principal)**
 - BP Internet Banking Platform
- **Naranja (Sistemas corporativos internos)**
 - Core Bancario
 - Sistema Complementario Cliente
- **Rojo (Servicios críticos externos)**
 - Identity Provider
 - Motor Biométrico
 - ACH/BCE
 - Pasarela de Pagos
- **Gris (Servicios de soporte)**
 - Email
 - SMS
 - Push
 - SIEM

Elementos del Diagrama

1. Personas

ID	Actor	Descripción
P1	Cliente Bancario	Usuario que consulta información, realiza pagos y transferencias
P2	Nuevo Cliente	Usuario que realiza onboarding digital
P3	Administrador Operacional	Supervisa operación y auditoría
P4	Equipo de Soporte	Monitorea salud y disponibilidad

2. Sistema Principal

ID	Sistema
S0	BP Internet Banking

Descripción:

Plataforma digital que permite a los clientes consultar productos, movimientos, realizar transferencias, pagos y procesos de onboarding digital.

3. Sistemas Externos

ID	Sistema	Descripción
S1	Core Bancario	Información de clientes, cuentas y movimientos
S2	Sistema Complementario Cliente	Información detallada del cliente
S3	Identity Provider	Autenticación OAuth2/OIDC
S4	Motor Biométrico	Validación facial y liveness
S5	Servicio Email	Notificaciones regulatorias
S6	Servicio SMS	Notificaciones regulatorias
S7	Servicio Push	Notificaciones móviles
S8	Red Interbancaria ACH/BCE	Transferencias interbancarias
S9	Pasarela de Pagos	Procesamiento de pagos
S10	Sistema SIEM / Auditoría Corporativa	Monitoreo y cumplimiento

Relaciones

Origen	Destino	Descripción
Cliente Bancario	BP Internet Banking	Consulta información y realiza transacciones
Nuevo Cliente	BP Internet Banking	Realiza onboarding digital
Administrador Operacional	BP Internet Banking	Supervisa operaciones
Equipo de Soporte	BP Internet Banking	Monitorea disponibilidad
BP Internet Banking	Core Bancario	Consulta productos, cuentas y movimientos
BP Internet Banking	Sistema Complementario Cliente	Consulta información ampliada

BP Internet Banking	Identity Provider	Autentica usuarios
BP Internet Banking	Motor Biométrico	Valida identidad durante onboarding
BP Internet Banking	Servicio Email	Envía notificaciones
BP Internet Banking	Servicio SMS	Envía notificaciones
BP Internet Banking	Servicio Push	Envía notificaciones
BP Internet Banking	ACH/BCE	Procesa transferencias interbancarias
BP Internet Banking	Pasarela de Pagos	Procesa pagos
BP Internet Banking	Sistema SIEM	Envía eventos de auditoría

Justificación Arquitectónica del Context Diagram

- **¿Por qué el Identity Provider aparece separado?**
Porque OAuth2/OIDC constituye un dominio independiente de seguridad y autenticación. Esto sigue ADR-006, ADR-007 y ADR-008.
- **¿Por qué el Motor Biométrico aparece separado?**
Porque el onboarding es un flujo distinto de la autenticación tradicional. Permite evolucionar o reemplazar tecnologías biométricas sin afectar al resto del ecosistema. Relacionado con ADR-008.
- **¿Por qué las notificaciones están desacopladas?**
Porque representan integraciones externas sujetas a disponibilidad independiente. Esto permite incorporar nuevos canales en el futuro. Relacionado con ADR-002.
- **¿Por qué SIEM aparece desde el Context Diagram?**
Porque auditoría y monitoreo son requerimientos explícitos del ejercicio. Además demuestra cumplimiento regulatorio y trazabilidad. Relacionado con ADR-014.

Relación con ADRs

Elemento	ADR
Identity Provider	ADR-006, ADR-007, ADR-008
Motor Biométrico	ADR-008
Notificaciones	ADR-002
Core Bancario	ADR-001
ACH/BCE	ADR-002
SIEM	ADR-014

4.3 Diagrama de Contenedores (Nivel 2)

Objetivo

El Diagrama de Contenedores representa la estructura interna de alto nivel del sistema BP Internet Banking.

Su propósito es mostrar cómo las responsabilidades funcionales son distribuidas entre aplicaciones, servicios, bases de datos y componentes tecnológicos, así como las principales interacciones entre ellos.

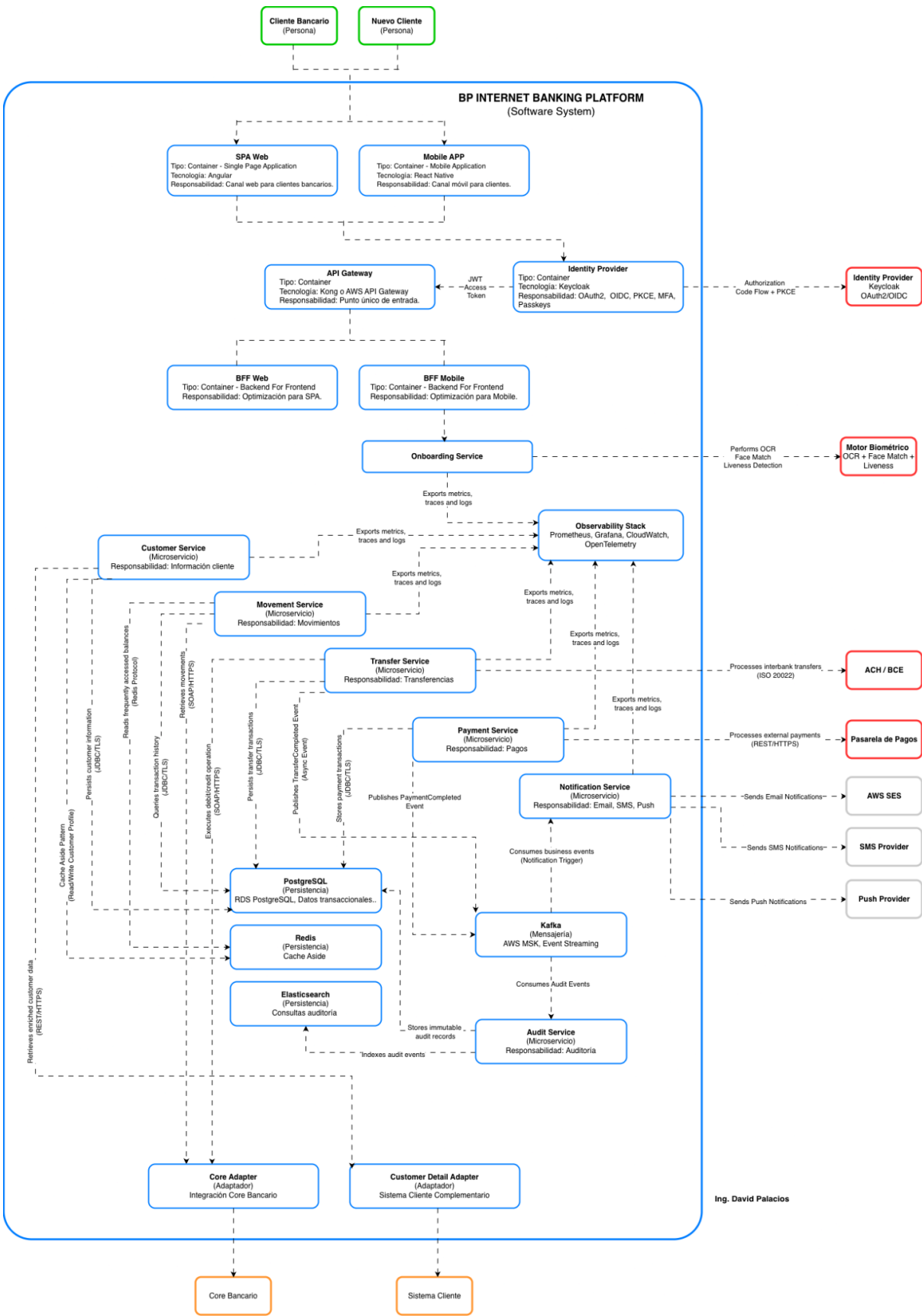
A diferencia del Diagrama de Contexto, este nivel sí incorpora tecnologías, mecanismos de integración, almacenamiento de datos y componentes de mensajería.

Este diagrama está orientado principalmente a:

- Arquitectos de Solución.
- Arquitectos Cloud.
- Líderes Técnicos.
- Desarrolladores.
- Equipos DevOps.
- Equipos de Operaciones.

La arquitectura propuesta sigue los principios definidos en los ADR previamente documentados, incluyendo microservicios, Event Driven Architecture, Backend For Frontend, OAuth 2.0, CQRS, Cache Aside y Observabilidad Distribuida.

Diagrama



Agrupación visual

- CAPA DE CANAL (CHANNEL LAYER)
 - SPA
 - Mobile
- CAPA DE ACCESO (ACCESS LAYER)
 - API Gateway
- CAPA DE EXPERIENCIA (EXPERIENCE LAYER)
 - BFF Web
 - BFF Mobile
- CAPA DE SEGURIDAD (SECURITY LAYER)
 - Identity Provider
- CAPA DE NEGOCIOS (BUSINESS LAYER)
 - Customer Service
 - Movement Service
 - Transfer Service
 - Payment Service
 - Notification Service
 - Audit Service
 - Onboarding Service
- CAPA DE INTEGRACIÓN (INTEGRATION LAYER)
 - Core Adapter
 - Customer Adapter
- CAPA DE DATOS (DATA LAYER)
 - Redis
 - PostgreSQL
 - Elasticsearch
 - Kafka
- CAPA DE OBSERVABILIDAD (OBSERVABILITY LAYER)
 - OpenTelemetry
 - Grafana
 - Prometheus
 - CloudWatch

Contenedores

Contenedor	Tecnología	Responsabilidad
SPA Web	Angular	Canal Web
Mobile App	React Native	Canal Móvil
API Gateway	Kong / AWS API Gateway	Entrada única
Identity Provider	Keycloak	OAuth2/OIDC
Customer Service	Spring Boot	Clientes
Movement Service	Spring Boot	Movimientos
Transfer Service	Spring Boot	Transferencias
Payment Service	Spring Boot	Pagos
Notification Service	Spring Boot	Notificaciones
Audit Service	Spring Boot	Auditoría
Kafka	AWS MSK	Eventos
Redis	ElastiCache	Caché
PostgreSQL	RDS PostgreSQL	Datos transaccionales
Elasticsearch	OpenSearch	Consultas auditoría

Relaciones

Origen	Destino	Propósito
SPA	Identity Provider	OAuth2/OIDC
Mobile	Identity Provider	OAuth2/OIDC
Transfer Service	Kafka	Publica eventos
Kafka	Notification Service	Consume eventos
Kafka	Audit Service	Consume eventos
Customer Service	Redis	Cache Aside
Transfer Service	ACH/BCE	Transferencias interbancarias
Payment Service	Pasarela de Pagos	Procesamiento pagos

Relación con ADRs

Contenedor	ADR
Angular SPA	ADR-009
React Native	ADR-010
Keycloak	ADR-008
OAuth2	ADR-006
PKCE	ADR-007
Kafka	ADR-002
PostgreSQL	ADR-011
Redis	ADR-012
CQRS	ADR-013
Observabilidad	ADR-014
Kubernetes	ADR-015

4.4 Diagrama de Componentes (Nivel 3)

Objetivo

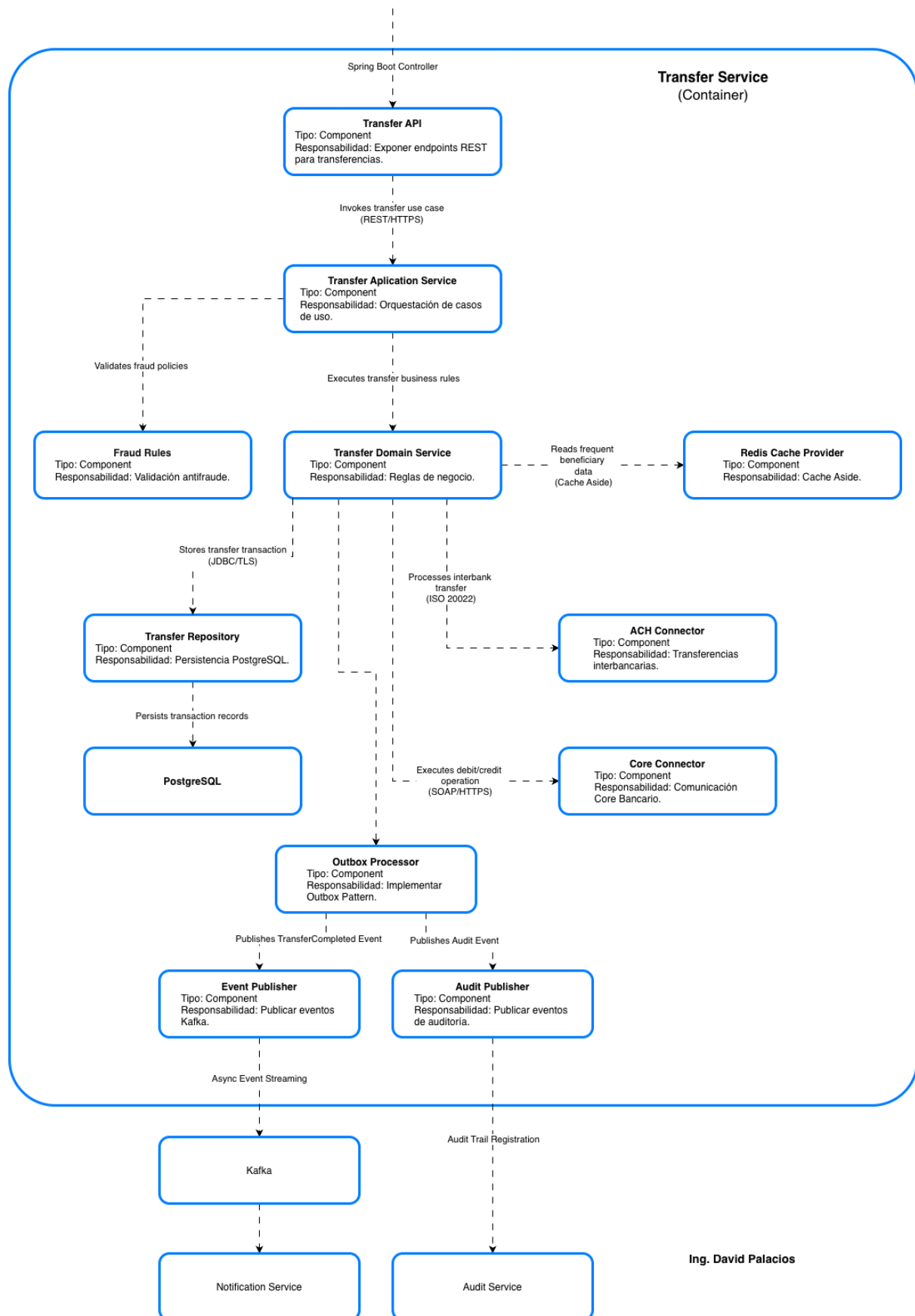
El Diagrama de Componentes representa una vista detallada de la estructura interna del contenedor Transfer Service.

Este nivel del modelo C4 permite identificar los componentes de software que implementan la lógica de negocio, sus responsabilidades, dependencias internas, mecanismos de integración y patrones arquitectónicos utilizados.

Debido a que el proceso de transferencias constituye una de las capacidades más críticas dentro de una plataforma bancaria, se seleccionó el contenedor Transfer Service para realizar el nivel de detalle requerido.

Este servicio participa en operaciones de transferencias entre cuentas propias e interbancarias, integra sistemas Core, interactúa con la red ACH/BCE, publica eventos de negocio y garantiza trazabilidad completa mediante mecanismos de auditoría.

Diagrama



Ing. David Palacios

Patrones aplicados

- CQRS Lite
- Cache Aside
- Outbox Pattern
- Adapter Pattern
- Anti-Corruption Layer
- Event Driven Architecture
- Circuit Breaker
- Retry Pattern

Relación con ADR

Componente	ADR
Redis Cache Provider	ADR-012
Outbox Processor	ADR-013
Event Publisher	ADR-002
PostgreSQL	ADR-011
Core Connector	ADR-001
Observability	ADR-014

5. Arquitectura de Seguridad y Autenticación

5.1 Objetivo

La arquitectura de seguridad tiene como objetivo garantizar la confidencialidad, integridad, disponibilidad y trazabilidad de la información financiera, cumpliendo con los estándares regulatorios aplicables al sector bancario.

La estrategia de seguridad propuesta sigue los principios de:

- Zero Trust Architecture.
- Least Privilege.
- Defense in Depth.
- Secure by Design.
- Security by Default.

5.2 Arquitectura de Autenticación

La solución adopta OAuth 2.0 y OpenID Connect (OIDC) como estándares de autenticación y autorización.

El Identity Provider seleccionado es Keycloak debido a su compatibilidad nativa con:

- OAuth 2.0
- OpenID Connect
- SAML 2.0
- MFA
- FIDO2
- Passkeys
- Integración biométrica

Justificación Arquitectónica

La autenticación constituye uno de los dominios más críticos dentro de una plataforma bancaria debido a que protege el acceso a información financiera sensible y a operaciones monetarias de alto impacto.

OAuth 2.0 fue seleccionado como estándar de autorización debido a su amplia adopción en la industria, compatibilidad con arquitecturas modernas basadas en APIs y soporte por parte de los principales proveedores de identidad.

OpenID Connect complementa OAuth 2.0 incorporando capacidades de autenticación estandarizadas mediante tokens de identidad (ID Tokens), permitiendo desacoplar completamente la gestión de identidad de las aplicaciones consumidoras.

La adopción de OAuth2/OIDC permite que la organización centralice políticas de autenticación, MFA, biometría, gestión de sesiones y revocación de credenciales sin modificar los canales digitales o microservicios.

Riesgos Mitigados

- Robo de credenciales.
- Exposición de contraseñas en aplicaciones.
- Reutilización de credenciales comprometidas.
- Falta de trazabilidad de accesos.

Beneficios

- Estándar ampliamente adoptado.
- Compatibilidad con Zero Trust.
- Integración sencilla con MFA y FIDO2.
- Escalabilidad organizacional.

5.3 Flujo de Autenticación Web

Flujo Seleccionado: Authorization Code Flow + PKCE

Alternativas Evaluadas:

- Implicit Flow
- Client Credentials
- Resource Owner Password Credentials
- Authorization Code Flow + PKCE

Decisión: Authorization Code Flow + PKCE

Justificación

- **Justificación 1:** OAuth 2.1 recomienda Authorization Code Flow con PKCE como mecanismo estándar para aplicaciones públicas, reemplazando completamente Implicit Flow debido a vulnerabilidades de exposición de tokens.
- **Justificación 2:** PKCE incorpora un código de verificación dinámico que impide ataques de interceptación del Authorization Code.
- **Justificación 3:** No requiere almacenar Client Secret en aplicaciones SPA o móviles, reduciendo riesgos de exposición de credenciales.

5.4 Flujo de Autenticación Mobile

La aplicación móvil utilizará:

- Authorization Code Flow + PKCE.
- MFA adaptativo.
- Biometría local.
- Passkeys.

El token de acceso será almacenado utilizando mecanismos seguros provistos por el sistema operativo:

- iOS Keychain.
- Android Keystore.

5.5 Multi-Factor Authentication (MFA)

Factores soportados:

- **Algo que conoce**
 - Usuario.
 - Contraseña.
 - PIN.
- **Algo que posee**
 - Dispositivo registrado.
 - Token OTP.
- **Algo que es**
 - Huella digital.
 - Face ID.
 - Reconocimiento facial.

5.6 Passkeys y FIDO2

La solución incorpora autenticación basada en FIDO2.

Beneficios:

- Eliminación de contraseñas débiles.
- Protección contra phishing.
- Protección contra credential stuffing.
- Mejor experiencia de usuario.

Los autenticadores soportados incluyen:

- Face ID.
- Touch ID.
- Windows Hello.
- Android Biometric Authentication.

5.7 Gestión de Sesiones

Las sesiones se gestionarán mediante JWT de corta duración.

Configuración propuesta:

- Access Token: 15 minutos
- Refresh Token: 8 horas

Controles:

- Rotación de Refresh Tokens.

- Revocación centralizada.
- Logout global.
- Detección de dispositivos comprometidos.
- Invalidación por riesgo.
- Device Fingerprinting.

5.8 Controles de Seguridad

- TLS 1.3.
- mTLS interno.
- WAF.
- Rate Limiting.
- API Security.
- Secrets Manager.
- IAM.
- KMS.
- GuardDuty.
- Security Hub.
- SIEM corporativo.

5.9 OWASP y Seguridad Aplicativa

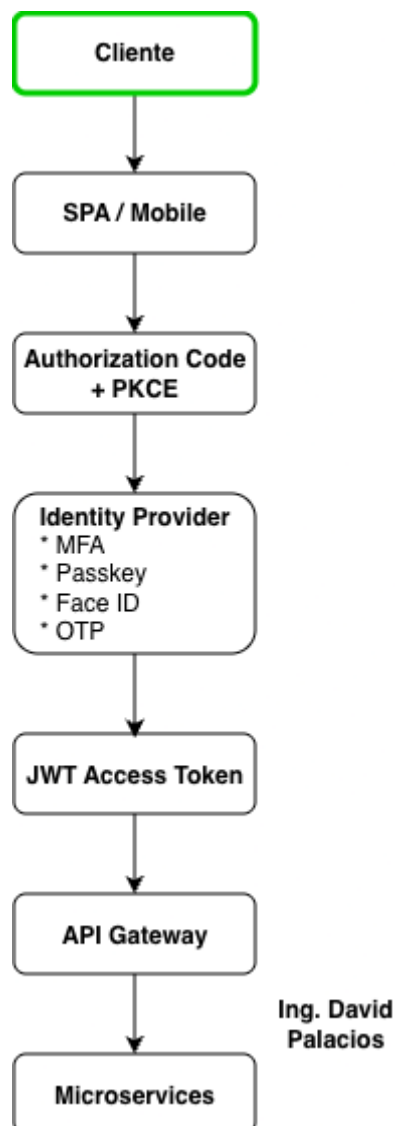
La solución seguirá:

- OWASP Top 10.
- OWASP ASVS.
- OWASP API Security Top 10.
- Secure SDLC.

Controles adicionales:

- SAST.
- DAST.
- Dependency Scanning.
- Container Scanning.
- Secret Scanning.

5.10 Diagrama de Flujo de Autenticación



6. Arquitectura de Onboarding Biométrico

6.1 Objetivo

Permitir la incorporación digital de nuevos clientes sin requerir presencia física en sucursales, garantizando cumplimiento regulatorio, prevención de fraude y validación robusta de identidad.

6.2 Flujo de Onboarding

- **Paso 1:** Captura del documento de identidad.
- **Paso 2:** OCR para extracción automática de información.
- **Paso 3:** Captura de selfie.

- **Paso 4:** Validación de prueba de vida (Liveness Detection).
- **Paso 5:** Comparación biométrica (Face Match).
- **Paso 6:** Validación contra listas regulatorias.
- **Paso 7:** Creación de identidad digital.
- **Paso 8:** Activación de usuario.
- **Paso 9:** Registro de dispositivo.
- **Paso 10:** Habilitación de acceso mediante biometría.

6.3 Componentes del Flujo

OCR Engine

Extrae automáticamente:

- Nombre.
- Documento.
- Fecha de nacimiento.
- Nacionalidad.

Face Match

Compara: Documento con Selfie

Validando la coincidencia biométrica.

Liveness Detection

Valida que la persona se encuentre físicamente presente y no se trate de:

- Fotografías.
- Videos.
- Deepfakes.
- Ataques de presentación.

Justificación Arquitectónica

Las entidades financieras enfrentan actualmente un incremento significativo en intentos de fraude digital, suplantación de identidad y creación fraudulenta de cuentas.

Por esta razón, el proceso de onboarding no debe limitarse únicamente a la captura de documentos de identidad, sino que debe incorporar mecanismos adicionales que permitan validar de forma robusta que la persona que solicita el alta es realmente quien dice ser.

La incorporación de OCR, Face Match y Liveness Detection permite construir un proceso de validación multicapa que reduce significativamente el riesgo de fraude documental y ataques de presentación. Adicionalmente, este enfoque mejora la experiencia del cliente al eliminar la necesidad de asistir físicamente a una sucursal bancaria para completar el proceso de incorporación.

Riesgos Mitigados

- Robo de identidad.
- Uso de documentos falsificados.
- Deepfakes.
- Fotografías impresas.
- Videos pregrabados.

Beneficios

- Incorporación digital completa.
- Reducción de fraude.
- Mejor experiencia de usuario.
- Menor costo operativo.
- Mayor escalabilidad comercial.

6.4 Herramientas Evaluadas

AWS Rekognition

Ventajas:

- Integración nativa AWS.
- Escalabilidad.
- Costos competitivos.

Desventajas:

- Requiere construir parte de la lógica.

Jumio

Ventajas:

- Solución integral KYC.
- OCR.
- Face Match.
- AML.

Desventajas:

- Mayor costo.

Onfido

Ventajas:

- Amplia adopción financiera.
- Excelente precisión.

Desventajas:

- Costos elevados.

FacePhi

Ventajas:

- Especialización bancaria.
- Alta precisión biométrica.

Desventajas:

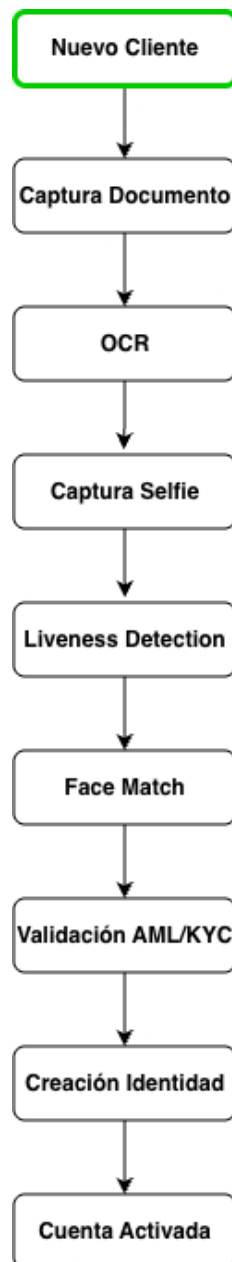
- Licenciamiento especializado.

6.5 Decisión

FacePhi o Jumio.

- **Justificación 1:** Ambas soluciones poseen amplia adopción en entidades financieras de Latinoamérica.
- **Justificación 2:** Reducen significativamente el esfuerzo de construcción y mantenimiento de capacidades biométricas.
- **Justificación 3:** Proporcionan mecanismos certificados de prevención de fraude digital y validación de identidad.

6.6 Diagrama de Proceso Onboarding Biométrico



Ing. David Palacios

7. Auditoría y Persistencia de Clientes Frecuentes

7.1 Arquitectura de Auditoría

La solución implementa un modelo basado en:

- Event Driven Architecture.
- Event Sourcing Parcial.
- Outbox Pattern.

Sin Outbox Pattern puede ocurrir que una transferencia sea almacenada exitosamente en la base de datos pero el evento correspondiente nunca sea publicado en Kafka debido a una falla de red o infraestructura.

Este escenario produciría inconsistencias operativas, auditorías incompletas y potenciales incumplimientos regulatorios. La utilización del patrón Outbox garantiza consistencia entre la persistencia de datos y la publicación de eventos, eliminando la posibilidad de pérdida de eventos críticos.

7.2 Flujo de Auditoría

Transfer Service

- Outbox Table
- Outbox Processor
- Kafka
- Audit Service
- Audit Database
- Elasticsearch

7.3 Justificación

- **Justificación 1:** Garantiza trazabilidad completa de todas las operaciones realizadas por los clientes.
- **Justificación 2:** Evita pérdida de eventos incluso ante fallos temporales de infraestructura.
- **Justificación 3:** Permite reconstruir operaciones históricas para fines regulatorios y forenses.

7.4 Persistencia de Clientes Frecuentes

Patrón Seleccionado: Cache Aside Pattern

Los sistemas Core bancarios suelen representar uno de los activos tecnológicos más críticos de la organización. Un acceso excesivo a dichos sistemas puede generar

degradación de rendimiento, incremento de costos operativos y afectación de otros procesos corporativos.

La implementación del patrón Cache Aside permite almacenar temporalmente información frecuentemente consultada, reduciendo la dependencia del Core y mejorando significativamente los tiempos de respuesta percibidos por el cliente.

Flujo:

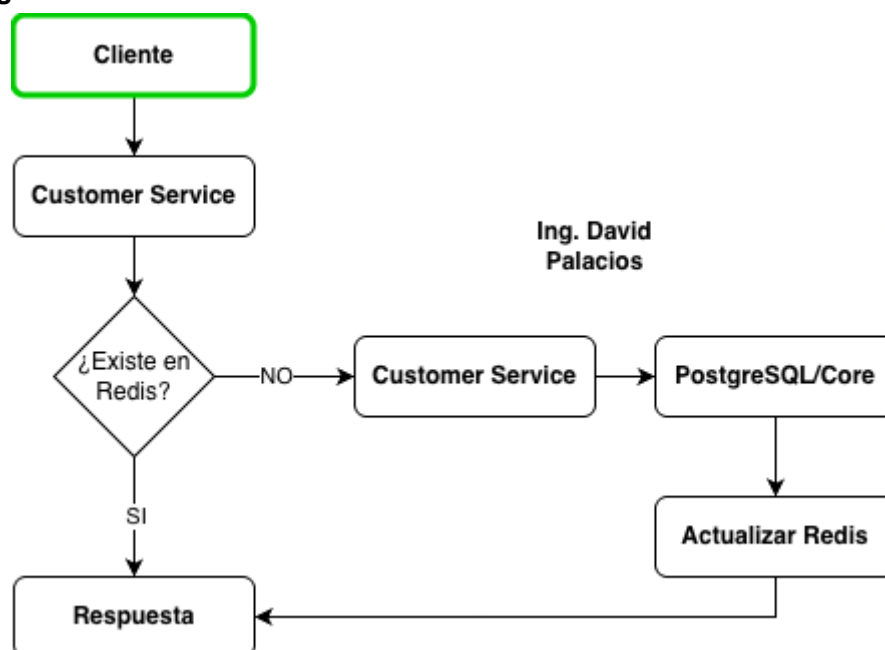
1. Cliente solicita información.
2. Customer Service consulta Redis.
3. Si existe información:
 - a. Responde inmediatamente.
4. Si no existe:
 - a. Consulta PostgreSQL/Core.
 - b. Actualiza Redis.
 - c. Responde.

Beneficios

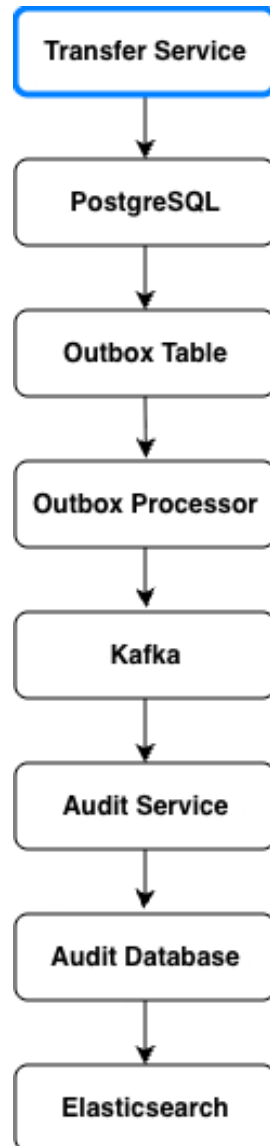
- **Menor Latencia:** Reduce significativamente los tiempos de respuesta.
- **Menor Carga sobre el Core:** Disminuye la cantidad de consultas realizadas a sistemas legados.
- **Escalabilidad:** Permite soportar altos volúmenes de consultas concurrentes.

7.5 Diagramas

- Diagrama Cache Aside



- Diagrama Outbox



Ing. David Palacios

8. Infraestructura AWS

Arquitectura Cloud: La solución se desplegará sobre AWS utilizando una estrategia Multi-AZ y Warm Standby.

La indisponibilidad de una plataforma bancaria puede generar pérdidas económicas, incumplimientos regulatorios y afectación reputacional.

Por esta razón, todos los componentes críticos se desplegarán en múltiples zonas de disponibilidad independientes. Ante la falla completa de una zona, los servicios continuarán operando desde las zonas restantes sin intervención manual, minimizando el impacto para los clientes finales.

Componentes Principales:

- Route53
- CloudFront
- AWS WAF
- Application Load Balancer
- Amazon EKS
- Amazon ECR
- Amazon MSK
- Amazon ElastiCache
- Amazon RDS PostgreSQL
- Amazon OpenSearch
- Amazon S3
- AWS KMS
- AWS Secrets Manager
- CloudWatch
- X-Ray
- GuardDuty
- Security Hub

Alta Disponibilidad (HA)

Componentes críticos desplegados en múltiples zonas de disponibilidad.

Objetivo: Disponibilidad $\geq 99.95\%$

Recuperación ante Desastres (DR)

Estrategia: Warm Standby

Se evaluaron tres estrategias:

- Backup & Restore
- Pilot Light
- Warm Standby

Se seleccionó Warm Standby porque proporciona un equilibrio adecuado entre costo, tiempo de recuperación y complejidad operativa para una plataforma bancaria de misión crítica.

Controles:

- Replicación Cross-Region.
- Backups automáticos.
- Snapshots.
- Pruebas periódicas de recuperación.

Objetivos:

- $RTO \leq 1$ hora
- $RPO \leq 15$ minutos

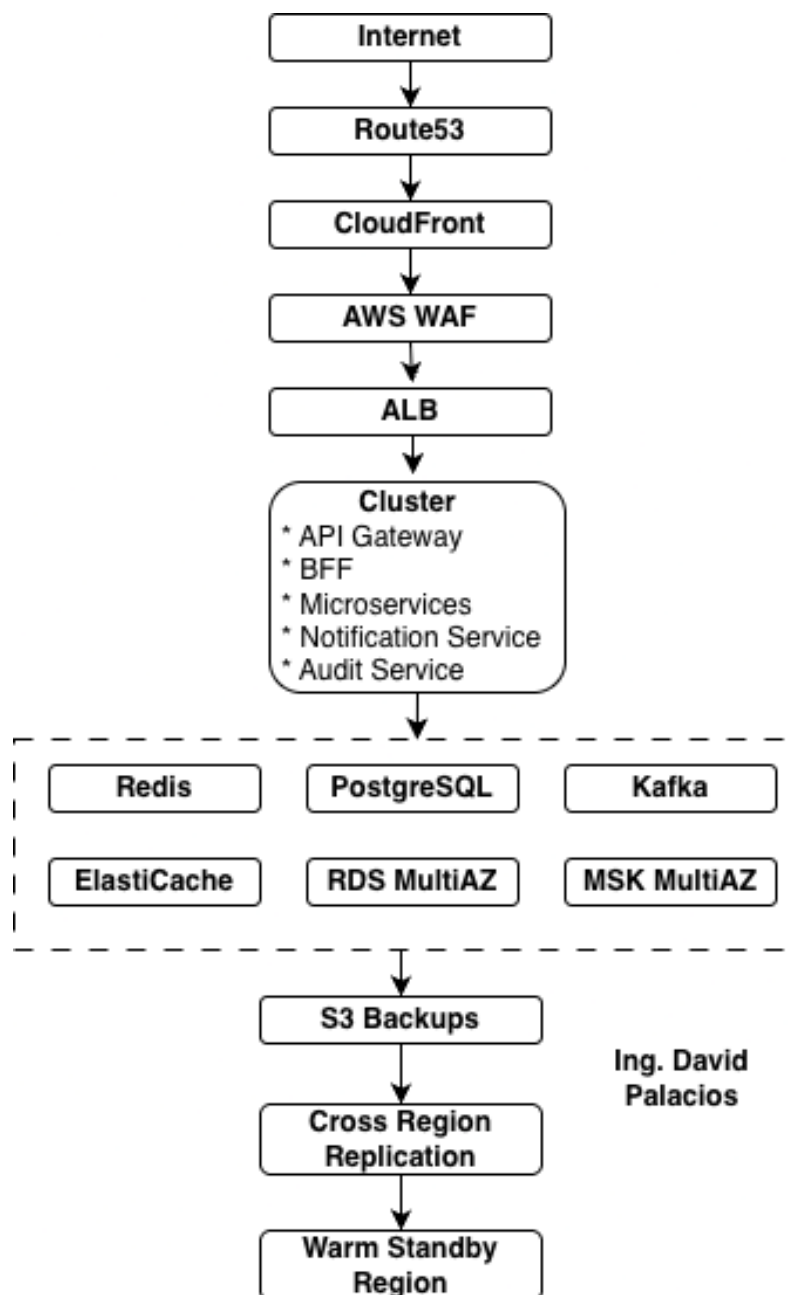
Auto-Healing

Implementado mediante Kubernetes (EKS).

Mecanismos:

- Liveness Probes.
- Readiness Probes.
- HPA.
- Cluster Autoscaler.
- Auto Recovery.
- Self-Healing Pods.

Diagrama AWS



Ing. David
Palacios

9. Regulaciones y Cumplimiento

9.1 Objetivo

La plataforma BP Internet Banking procesará información financiera y datos personales sensibles de los clientes, por lo que deberá cumplir con regulaciones nacionales e internacionales relacionadas con protección de datos, continuidad del negocio, seguridad de la información, gestión de riesgos y trazabilidad de operaciones.

El cumplimiento regulatorio no debe considerarse únicamente como una obligación legal, sino como un habilitador de confianza, resiliencia operativa y sostenibilidad tecnológica para la organización.

9.2 Marco Regulatorio Aplicable

Ley Orgánica de Protección de Datos Personales (Ecuador)

La solución deberá garantizar:

- Consentimiento explícito para el tratamiento de datos.
- Derecho de acceso, rectificación y eliminación.
- Protección de datos biométricos.
- Registro de actividades de tratamiento.
- Trazabilidad de accesos y modificaciones.

PCI DSS

Aplicable a los procesos relacionados con pagos y manejo de información financiera.

Controles principales:

- Cifrado de información sensible.
- Segmentación de redes.
- Gestión de accesos privilegiados.
- Monitoreo continuo.
- Gestión de vulnerabilidades.

ISO 27001

La arquitectura se alinea con controles de:

- Gestión de riesgos.
- Seguridad de la información.
- Gestión de incidentes.
- Gestión de activos.
- Control de accesos.

ISO 22301

Garantiza la continuidad operativa mediante:

- Planes de recuperación.
- Estrategias de contingencia.
- Pruebas periódicas.
- Gestión de crisis.

NIST Cybersecurity Framework

Se adopta como referencia para:

- Identificar.
- Proteger.
- Detectar.
- Responder.
- Recuperar.

9.3 Estándares Técnicos de Seguridad

- **OAuth 2.0:** Estándar de autorización para aplicaciones web y móviles.
- **OpenID Connect (OIDC):** Capa de autenticación sobre OAuth2.
- **FIDO2:** Autenticación resistente a phishing mediante biometría y Passkeys.
- **OWASP ASVS:** Framework de verificación de seguridad utilizado para validar la aplicación durante el ciclo de vida de desarrollo.
- **OWASP Top 10:** Base para el diseño seguro y mitigación de vulnerabilidades comunes.
- **CIS Controls:** Conjunto de controles para endurecimiento de infraestructura y operación segura.

9.4 Gobierno y Gestión de Riesgos

La solución incorpora controles de gobierno para garantizar que los riesgos tecnológicos sean identificados, evaluados y mitigados de manera continua.

Principales controles:

- Gestión de cambios.
- Gestión de vulnerabilidades.
- Gestión de accesos privilegiados.
- Gestión de incidentes.
- Gestión de configuración.
- Gestión de secretos.
- Gestión de evidencias de auditoría.

9.5 Cumplimiento y Trazabilidad

Toda operación realizada por un usuario deberá generar evidencia auditable incluyendo:

- Usuario.
- Fecha y hora.
- Dirección IP.
- Dispositivo.
- Operación ejecutada.
- Resultado de la operación.
- Correlation ID.

Esta información permitirá cumplir requerimientos regulatorios, investigaciones forenses y procesos de auditoría interna o externa.

9.6 Justificación Arquitectónica

La adopción de estándares internacionales reduce riesgos regulatorios, facilita auditorías externas y proporciona una base sólida para futuras certificaciones.

Adicionalmente, permite que la plataforma evolucione manteniendo alineación con prácticas reconocidas globalmente dentro del sector financiero.

9.7 Riesgos Mitigados

- Incumplimientos regulatorios.
- Sanciones legales.
- Exposición de datos personales.
- Fraude.
- Pérdida de evidencias de auditoría.
- Interrupción de servicios críticos.

9.8 Beneficios

- Cumplimiento normativo.
- Confianza del cliente.
- Reducción de riesgos operativos.
- Mayor resiliencia organizacional.
- Facilidad de auditoría y supervisión.

10. Operación, Monitoreo y DevOps

10.1 Objetivo

Garantizar la operación continua de la plataforma mediante observabilidad integral, automatización de despliegues, detección temprana de incidentes y mecanismos de recuperación automática.

La estrategia operacional busca maximizar disponibilidad, reducir tiempo de recuperación y mejorar la experiencia de clientes y equipos de soporte.

10.2 Observabilidad

La observabilidad se implementará mediante tres pilares fundamentales:

Métricas

Permiten medir comportamiento y rendimiento del sistema.

Ejemplos:

- Latencia.
- TPS (Transactions Per Second).

- Uso de CPU.
- Uso de memoria.
- Throughput.

Logs

Permiten analizar eventos y errores. Características:

- Centralizados.
- Estructurados.
- Correlacionados.

Trazas Distribuidas

Permiten seguir una transacción extremo a extremo entre microservicios.

10.3 Herramientas de Monitoreo

- **Amazon CloudWatch:** Monitoreo nativo de AWS.
- **Prometheus:** Recolección de métricas.
- **Grafana:** Visualización y dashboards.
- **OpenTelemetry:** Instrumentación estándar para métricas, logs y trazas.
- **AWS X-Ray:** Análisis de trazas distribuidas.
- **PagerDuty:** Gestión de incidentes y alertamiento.

10.4 Indicadores Operativos

SLI (Service Level Indicators)

- Disponibilidad.
- Latencia.
- Tasa de errores.
- Saturación.

SLO (Service Level Objectives)

- **Disponibilidad objetivo:** 99.95%
- **Latencia promedio:** < 300 ms para operaciones de consulta.

SLA (Service Level Agreement)

Compromiso contractual de disponibilidad y continuidad del servicio.

10.5 Gestión de Incidentes

La organización deberá contar con:

- Procedimiento de gestión de incidentes.
- Runbooks operativos.
- Escalamiento técnico.
- Postmortems.
- Gestión de problemas.

10.6 Estrategia DevOps

La solución adopta principios DevOps para acelerar entregas y reducir riesgos operativos.

Objetivos:

- Automatización.
- Calidad continua.
- Entrega continua.
- Recuperación rápida.

10.7 Pipeline CI/CD

Herramientas propuestas:

- **GitHub**: Gestión de código fuente.
- **GitHub Actions**: Automatización de pipelines.
- **Amazon ECR**: Registro de imágenes Docker.
- **Amazon EKS**: Orquestación de contenedores.

Flujo CI/CD

- Developer
 - Pull Request
 - Code Review
 - Build
 - Unit Tests
 - Security Scans
 - Docker Build
 - ECR
 - Deploy EKS
 - Validation
 - Production

10.8 Controles de Calidad

- **SonarQube**: Análisis de calidad de código.
- **SAST**: Análisis estático de vulnerabilidades.
- **DAST**: Pruebas dinámicas de seguridad.
- **Dependency Scanning**: Detección de librerías vulnerables.
- **Container Scanning**: Detección de vulnerabilidades en imágenes.

10.9 Estrategias de Despliegue

- **Blue-Green Deployment**: Permite cambios con mínimo riesgo.
- **Canary Deployment**: Permite validar cambios con una pequeña porción de usuarios antes de liberar completamente.
- **Rolling Updates**: Actualizaciones progresivas controladas.

10.10 Infraestructura como Código (IaC)

Herramienta seleccionada: Terraform.

Beneficios:

- Reproducibilidad.
- Trazabilidad.
- Automatización.
- Menor error humano.

10.11 Justificación Arquitectónica

La observabilidad integral y la automatización permiten detectar problemas antes de que impacten a los usuarios finales, reduciendo significativamente el MTTR (Mean Time To Recovery).

La adopción de DevOps acelera la entrega de valor al negocio sin sacrificar seguridad ni estabilidad operativa.

10.12 Beneficios

- Mayor disponibilidad.
- Menor MTTR.
- Despliegues seguros.
- Menor riesgo operativo.
- Mayor productividad de los equipos.

11. Costos y Optimización

11.1 Objetivo

Diseñar una arquitectura escalable y resiliente que optimice la relación costo-beneficio sin comprometer disponibilidad, seguridad ni cumplimiento regulatorio.

11.2 Principios de Optimización

La estrategia de costos se basa en:

- Uso eficiente de recursos.
- Escalabilidad bajo demanda.
- Servicios administrados.
- Automatización operativa.
- Pago por consumo.

11.3 Justificación de Servicios Administrados

Se priorizaron servicios administrados debido a que reducen significativamente el esfuerzo operativo asociado a mantenimiento, actualización, monitoreo y recuperación.

Ejemplos:

- Amazon RDS.
- Amazon MSK.
- Amazon ElastiCache.
- Amazon EKS.
- AWS WAF.

11.4 Estrategias de Optimización

Auto Scaling

Los recursos aumentan o disminuyen automáticamente según la demanda.

Beneficios:

- Reducción de costos.
- Mejor utilización.
- Escalabilidad automática.

Ambientes Independientes

Separación entre:

- Desarrollo.
- QA.
- Producción.

Permite optimizar recursos según criticidad.

Uso de Contenedores

Los contenedores permiten una utilización más eficiente de infraestructura en comparación con servidores dedicados.

Almacenamiento por Ciclo de Vida

Políticas de Lifecycle Management en S3:

- Standard.
- Standard IA.
- Glacier.
- Deep Archive.

11.5 Evaluación de Alternativas

Opción 1: Infraestructura Tradicional

Ventajas:

- Control total.

Desventajas:

- Mayor complejidad operativa.
- Mayor costo de mantenimiento.
- Menor elasticidad.

Opción 2: Servicios Administrados AWS

Ventajas:

- Menor carga operativa.
- Alta disponibilidad integrada.
- Escalabilidad automática.

Desventajas:

- Mayor dependencia del proveedor.

11.6 FinOps y Gobierno de Costos

La organización deberá implementar prácticas FinOps:

- Etiquetado de recursos.
- Presupuestos.
- Alertas de consumo.
- Dashboards de costos.
- Optimización continua.

Herramientas:

- AWS Cost Explorer.
- AWS Budgets.
- AWS Trusted Advisor.

11.7 Balance entre Costo y Disponibilidad

Se evaluaron diferentes estrategias de recuperación ante desastres:

Estrategia	Costo	RTO	RPO
Backup & Restore	Bajo	Alto	Alto
Pilot Light	Medio	Medio	Medio
Warm Standby	Medio-Alto	Bajo	Bajo
Active-Active	Muy Alto	Muy Bajo	Muy Bajo

La estrategia seleccionada fue Warm Standby debido a que ofrece el mejor equilibrio entre disponibilidad, resiliencia y costo para una plataforma bancaria de misión crítica.

11.8 Justificación Arquitectónica

La arquitectura propuesta prioriza la automatización, elasticidad y uso de servicios administrados para maximizar el valor entregado al negocio.

Aunque algunas decisiones pueden representar un costo inicial superior frente a soluciones tradicionales, reducen significativamente riesgos operativos, tiempos de indisponibilidad y costos de mantenimiento a largo plazo.

11.9 Beneficios

- Optimización continua.
- Escalabilidad eficiente.
- Menor costo operativo.
- Mayor previsibilidad financiera.
- Mejor aprovechamiento de recursos cloud.

12. Matriz de Cobertura de Requerimientos

12.1 Objetivo

La presente matriz permite verificar la trazabilidad entre los requerimientos planteados en el ejercicio técnico y los componentes de la solución arquitectónica propuesta.

Su propósito es demostrar que todos los requerimientos funcionales, no funcionales, regulatorios y tecnológicos fueron considerados durante el diseño de la arquitectura.

12.2 Cobertura de Requerimientos Funcionales

Requerimiento	Solución Propuesta	Sección
Consulta histórica de movimientos	Movement Service + Core Adapter + Redis Cache	4.3, 4.4, 7
Consulta de productos y cuentas	Customer Service + Core Adapter	4.3, 4.4
Transferencias entre cuentas propias	Transfer Service	4.3, 4.4
Transferencias interbancarias	Transfer Service + ACH Connector	4.3, 4.4
Procesamiento de pagos	Payment Service + Pasarela de Pagos	4.3
Notificaciones regulatorias	Notification Service + Email + SMS + Push	4.3
Onboarding digital	Onboarding Service + Motor Biométrico	6
Autenticación de usuarios	OAuth2 + OIDC + Keycloak	5
Auditoría de operaciones	Audit Service + Kafka + Outbox Pattern	7
Persistencia de clientes frecuentes	Redis + Cache Aside Pattern	7

12.3 Cobertura de Requerimientos No Funcionales

Requerimiento	Solución Propuesta	Sección
Alta disponibilidad	Multi-AZ + EKS + RDS Multi-AZ + MSK Multi-AZ	8
Escalabilidad	Kubernetes + Auto Scaling + Microservicios	3, 8
Recuperación ante desastres	Warm Standby + Replicación Cross-Region	8
Seguridad	OAuth2, OIDC, MFA, FIDO2, WAF, KMS	5
Observabilidad	OpenTelemetry + Grafana + CloudWatch + X-Ray	10
Auditoría completa	Event Sourcing Parcial + Outbox + Elasticsearch	7
Tolerancia a fallos	Circuit Breaker + Retry + Bulkhead + Multi-AZ	3, 8
Baja latencia	Redis Cache + CloudFront	7, 8
Auto-Healing	Kubernetes + HPA + Readiness/Liveness Probes	8
Excelencia Operativa	DevOps + CI/CD + IaC	10

12.4 Cobertura de Requerimientos Tecnológicos

Requerimiento	Solución Propuesta
Arquitectura desacoplada	Microservicios + Kafka
Integración con sistemas externos	API Gateway + Adapters
Front-End Web	Angular
Aplicación móvil multiplataforma	React Native
Integración biométrica	FacePhi / Jumio
Base de datos transaccional	PostgreSQL
Persistencia distribuida	Redis
Mensajería	Kafka (MSK)
Observabilidad	OpenTelemetry
Infraestructura Cloud	AWS

12.5 Cobertura de Criterios de Evaluación

Criterio de Evaluación	Cobertura
Calidad y profundidad de diagramas C4	Cumplido
Segmentación de responsabilidades	Cumplido
Uso de patrones arquitectónicos	Cumplido
Integración con servicios externos	Cumplido
Arquitectura Front-End	Cumplido
Arquitectura Mobile	Cumplido
Arquitectura de acceso a datos	Cumplido
Conocimientos Cloud AWS	Cumplido
Manejo de costos	Cumplido
Arquitectura de autenticación	Cumplido
Arquitectura de onboarding	Cumplido
Arquitectura de auditoría	Cumplido
Regulaciones y seguridad	Cumplido
Alta disponibilidad	Cumplido
Tolerancia a fallos	Cumplido
Monitoreo y observabilidad	Cumplido

12.6 Resultado de Validación

Tras la revisión integral de la solución propuesta, se concluye que todos los requerimientos establecidos en el ejercicio técnico fueron cubiertos mediante componentes arquitectónicos específicos, mecanismos de seguridad, integración, observabilidad, resiliencia y gobierno tecnológico.

La arquitectura presentada satisface tanto los objetivos funcionales como los atributos de calidad esperados para una plataforma bancaria moderna orientada a escalabilidad, seguridad y evolución futura.

13. Conclusiones Arquitectónicas

La solución propuesta para BP Internet Banking fue diseñada siguiendo principios de arquitectura moderna orientados a resiliencia, escalabilidad, seguridad y mantenibilidad.

La arquitectura basada en microservicios permite desacoplar los dominios funcionales principales, facilitando la evolución independiente de capacidades como transferencias, pagos, onboarding, autenticación y notificaciones.

La incorporación de mecanismos de integración mediante API Gateway, Kafka y patrones como Adapter, Outbox y Cache Aside proporciona una plataforma flexible capaz de integrarse con sistemas Core existentes sin comprometer la estabilidad operativa.

Desde la perspectiva de seguridad, la adopción de OAuth 2.0, OpenID Connect, MFA, FIDO2 y biometría permite implementar controles robustos alineados con las mejores prácticas de la industria financiera y los principios de Zero Trust.

La estrategia de observabilidad, auditoría y monitoreo garantiza trazabilidad completa de las operaciones, facilitando tanto el cumplimiento regulatorio como la operación diaria de la plataforma.

En términos de continuidad de negocio, la utilización de AWS como plataforma cloud, junto con despliegues Multi-AZ, recuperación ante desastres mediante Warm Standby y mecanismos de Auto-Healing sobre Kubernetes, proporciona una infraestructura preparada para soportar cargas críticas y escenarios de falla.

Finalmente, la solución fue diseñada considerando extensibilidad futura, permitiendo incorporar nuevos productos financieros, canales digitales o integraciones externas sin requerir rediseños significativos de la plataforma.

En consecuencia, se concluye que la arquitectura propuesta cumple los objetivos funcionales, tecnológicos, operativos y regulatorios planteados en el ejercicio, proporcionando una base sólida para la construcción de una plataforma bancaria digital moderna, segura y escalable.

14. Referencias y Fuentes Consultadas

La arquitectura propuesta fue elaborada tomando como referencia estándares, marcos de arquitectura, documentación técnica oficial y mejores prácticas ampliamente adoptadas en la industria financiera y tecnológica.

Arquitectura y Diseño

- C4 Model – Simon Brown.
- Software Architecture for Developers – Simon Brown.
- Microservices Patterns – Chris Richardson.
- Building Evolutionary Architectures – Neal Ford, Rebecca Parsons y Patrick Kua.

Seguridad

- OAuth 2.0 Authorization Framework (RFC 6749).
- OAuth 2.1 Draft Specification.
- OpenID Connect Core Specification.
- FIDO2 Web Authentication (WebAuthn).
- OWASP Application Security Verification Standard (ASVS).
- OWASP Top 10.
- OWASP API Security Top 10.

Continuidad y Resiliencia

- AWS Well-Architected Framework.
- AWS Reliability Pillar.
- AWS Security Pillar.
- AWS Operational Excellence Pillar.

Observabilidad y Operaciones

- OpenTelemetry Documentation.
- Prometheus Documentation.
- Grafana Documentation.
- Kubernetes Documentation.

Normativas y Cumplimiento

- Ley Orgánica de Protección de Datos Personales (Ecuador).
- PCI DSS.
- ISO/IEC 27001.
- ISO 22301.
- NIST Cybersecurity Framework.
- CIS Controls.

Servicios Cloud

- AWS Architecture Center.
- AWS Reference Architectures.
- Amazon EKS Documentation.
- Amazon RDS Documentation.
- Amazon MSK Documentation.
- Amazon ElastiCache Documentation.

Estas referencias fueron utilizadas como base conceptual para la toma de decisiones arquitectónicas, selección tecnológica y definición de controles de seguridad, resiliencia y operación descritos a lo largo del documento.

